PY16CHESS SUNFISH

PY-16 FANTASY CONSOLE CARTRIDGE

BY PROF PLANLOZ

2026-05-30

OPEN THIS PDF WITH PY-16 TO PLAY

PY16CHESS SUNFISH - MANUAL

DESCRIPTION

PY-16 CHESS (Couch Co-op Edition)
A complete 16-bit chess game for py-16.
Play locally against a friend (with two gamepads)
or challenge the built-in chess AI.
Includes a chess clock, save system, and customizable colors!

PY-16 CHESS (Couch Co-op Edition)
A complete 16-bit chess game for py-16.
Play locally against a friend (with two gamepads)
or challenge the built-in chess AI.
Includes a chess clock, save system, and customizable colors!

CONTROLS

```
D-Pad / Arrows    : Move cursor
Z / A-Button      : Select piece / Execute move
Mouse / Left click: Alternative controls (Point & Click)
S / A (Keyboard)  : Save / Load game
Space / Select    : Back to menu / Cancel game
```

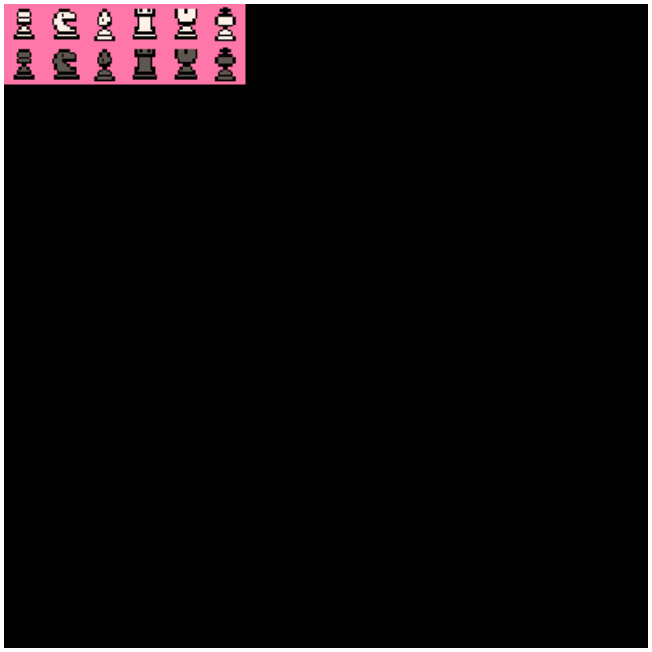
```
D-Pad / Arrows    : Move cursor
Z / A-Button      : Select piece / Execute move
Mouse / Left click: Alternative controls (Point & Click)
S / A (Keyboard)  : Save / Load game
Space / Select    : Back to menu / Cancel game
```

CREDITS

Code, Graphics & UI : Claude, Gemini, ProfPlanloz
AI-Engine : Sunfish (Thomas Ahle)
License : GNU General Public License 3(GPLv3)
Code, Graphics & UI : py-16 Gamedev
AI-Engine : Sunfish (Thomas Ahle)

PY16CHESS SUNFISH - ASSETS

SPRITE SHEET (256x256)



MAP (128x128)

SFX SLOTS

MUSIC TRACKS

```
# @manual
# @description
# PY-16 CHESS (Couch Co-op Edition)
# A complete 16-bit chess game for py-16.
# Play locally against a friend (with two gamepads)
# or challenge the built-in chess AI.
# Includes a chess clock, save system, and customizable colors!
#
# @controls
# D-Pad / Arrows    : Move cursor
# Z / A-Button      : Select piece / Execute move
# Mouse / Left click: Alternative controls (Point & Click)
# S / A (Keyboard)  : Save / Load game
# Space / Select    : Back to menu / Cancel game
#
# @credits
# Code, Graphics & UI : Claude, Gemini, ProfPlanloz
# AI-Engine           : Sunfish (Thomas Ahle)
# License             : GNU General Public License 3(GPLv3)
# @end

import py16
import os
import json

import time
from itertools import count
from collections import namedtuple
import sys as _sys
_sys.setrecursionlimit(10000)

# ===============================================================================
#  Embedded Sunfish engine - (c) Thomas Dybdahl Ahle, GPLv3.
#  Source: github.com/thomasahle/sunfish (sunfish.py, 'sunfish 2023').
#  Only the engine core (piece tables, Position, Searcher) is included; the
#  UCI front-end is stripped. The cart drives it via the bridge below.
# ===============================================================================

piece = {"P": 100, "N": 280, "B": 320, "R": 479, "Q": 929, "K": 60000}
pst = {
    'P': (   0,   0,   0,   0,   0,   0,   0,   0,
            78,  83,  86,  73, 102,  82,  85,  90,
             7,  29,  21,  44,  40,  31,  44,   7,
           -17,  16,  -2,  15,  14,   0,  15, -13,
           -26,   3,  10,   9,   6,   1,   0, -23,
           -22,   9,   5, -11, -10,  -2,   3, -19,
           -31,   8,  -7, -37, -36, -14,   3, -31,
             0,   0,   0,   0,   0,   0,   0,   0),
    'N': ( -66, -53, -75, -75, -10, -55, -58, -70,
            -3,  -6, 100, -36,   4,  62,  -4, -14,
            10,  67,   1,  74,  73,  27,  62,  -2,
            24,  24,  45,  37,  33,  41,  25,  17,
            -1,   5,  31,  21,  22,  35,   2,   0,
           -18,  10,  13,  22,  18,  15,  11, -14,
           -23, -15,   2,   0,   2,   0, -23, -20,
           -74, -23, -26, -24, -19, -35, -22, -69),
    'B': ( -59, -78, -82, -76, -23,-107, -37, -50,
           -11,  20,  35, -42, -39,  31,   2, -22,
            -9,  39, -32,  41,  52, -10,  28, -14,
            25,  17,  20,  34,  26,  25,  15,  10,
            13,  10,  17,  23,  17,  16,   0,   7,
            14,  25,  24,  15,   8,  25,  20,  15,
            19,  20,  11,   6,   7,   6,  20,  16,
            -7,   2, -15, -12, -14, -15, -10, -10),
```

```
    'R': (  35,  29,  33,   4,  37,  33,  56,  50,
            55,  29,  56,  67,  55,  62,  34,  60,
            19,  35,  28,  33,  45,  27,  25,  15,
             0,   5,  16,  13,  18,  -4,  -9,  -6,
           -28, -35, -16, -21, -13, -29, -46, -30,
           -42, -28, -42, -25, -25, -35, -26, -46,
           -53, -38, -31, -26, -29, -43, -44, -53,
           -30, -24, -18,   5,  -2, -18, -31, -32),
    'Q': (   6,   1,  -8,-104,  69,  24,  88,  26,
            14,  32,  60, -10,  20,  76,  57,  24,
            -2,  43,  32,  60,  72,  63,  43,   2,
             1, -16,  22,  17,  25,  20, -13,  -6,
           -14, -15,  -2,  -5,  -1, -10, -20, -22,
           -30,  -6, -13, -11, -16, -11, -16, -27,
           -36, -18,   0, -19, -15, -15, -21, -38,
           -39, -30, -31, -13, -31, -36, -34, -42),
    'K': (   4,  54,  47, -99, -99,  60,  83, -62,
           -32,  10,  55,  56,  56,  55,  10,   3,
           -62,  12, -57,  44, -67,  28,  37, -31,
           -55,  50,  11,  -4, -19,  13,   0, -49,
           -55, -43, -52, -28, -51, -47,  -8, -50,
           -47, -42, -43, -79, -64, -32, -29, -32,
            -4,   3, -14, -50, -57, -18,  13,   4,
            17,  30,  -3, -14,   6,  -1,  40,  18),
}
# Pad tables and join piece and pst dictionaries
for k, table in pst.items():
    padrow = lambda row: (0,) + tuple(x + piece[k] for x in row) + (0,)
    pst[k] = sum((padrow(table[i * 8 : i * 8 + 8]) for i in range(8)), ())
    pst[k] = (0,) * 20 + pst[k] + (0,) * 20

############################################################################
# Global constants
############################################################################

# Our board is represented as a 120 character string. The padding allows for
# fast detection of moves that don't stay within the board.
A1, H1, A8, H8 = 91, 98, 21, 28
initial = (
    "         \n"  #   0 -  9
    "         \n"  #  10 - 19
    " rnbqkbnr\n"  #  20 - 29
    " pppppppp\n"  #  30 - 39
    " ........\n"  #  40 - 49
    " ........\n"  #  50 - 59
    " ........\n"  #  60 - 69
    " ........\n"  #  70 - 79
    " PPPPPPPP\n"  #  80 - 89
    " RNBQKBNR\n"  #  90 - 99
    "         \n"  # 100 -109
    "         \n"  # 110 -119
)

# Lists of possible moves for each piece type.
N, E, S, W = -10, 1, 10, -1
directions = {
    "P": (N, N+N, N+W, N+E),
    "N": (N+N+E, E+N+E, E+S+E, S+S+E, S+S+W, W+S+W, W+N+W, N+N+W),
    "B": (N+E, S+E, S+W, N+W),
    "R": (N, E, S, W),
    "Q": (N, E, S, W, N+E, S+E, S+W, N+W),
    "K": (N, E, S, W, N+E, S+E, S+W, N+W)
}

```

```
# Mate value must be greater than 8*queen + 2*(rook+knight+bishop)
# King value is set to twice this value such that if the opponent is
# 8 queens up, but we got the king, we still exceed MATE_VALUE.
# When a MATE is detected, we'll set the score to MATE_UPPER - plies to get there
# E.g. Mate in 3 will be MATE_UPPER - 6
MATE_LOWER = piece["K"] - 10 * piece["Q"]
MATE_UPPER = piece["K"] + 10 * piece["Q"]

# Constants for tuning search
QS = 40
QS_A = 140
EVAL_ROUGHNESS = 15



################################################################################
# Chess logic
################################################################################


Move = namedtuple("Move", "i j prom")


class Position(namedtuple("Position", "board score wc bc ep kp")):
    """A state of a chess game
    board -- a 120 char representation of the board
    score -- the board evaluation
    wc -- the castling rights, [west/queen side, east/king side]
    bc -- the opponent castling rights, [west/king side, east/queen side]
    ep - the en passant square
    kp - the king passant square
    """

    def gen_moves(self):
        # For each of our pieces, iterate through each possible 'ray' of moves,
        # as defined in the 'directions' map. The rays are broken e.g. by
        # captures or immediately in case of pieces such as knights.
        for i, p in enumerate(self.board):
            if not p.isupper():
                continue
            for d in directions[p]:
                for j in count(i + d, d):
                    q = self.board[j]
                    # Stay inside the board, and off friendly pieces
                    if q.isspace() or q.isupper():
                        break
                    # Pawn move, double move and capture
                    if p == "P":
                        if d in (N, N + N) and q != ".": break
                        if d == N + N and (i < A1 + N or self.board[i + N] != "."): break
                        if (
                            d in (N + W, N + E)
                            and q == "."
                            and j not in (self.ep, self.kp, self.kp - 1, self.kp + 1)
                            #and j != self.ep and abs(j - self.kp) >= 2
                        ):
                            break
                        # If we move to the last row, we can be anything
                        if A8 <= j <= H8:
                            for prom in "NBRQ":
                                yield Move(i, j, prom)
                            break
                    # Move it
                    yield Move(i, j, "")
```

```
                    # Stop crawlers from sliding, and sliding after captures
                    if p in "PNK" or q.islower():
                        break
                    # Castling, by sliding the rook next to the king
                    if i == A1 and self.board[j + E] == "K" and self.wc[0]:
                        yield Move(j + E, j + W, "")
                    if i == H1 and self.board[j + W] == "K" and self.wc[1]:
                        yield Move(j + W, j + E, "")

    def rotate(self, nullmove=False):
        """Rotates the board, preserving enpassant, unless nullmove"""
        return Position(
            self.board[::-1].swapcase(), -self.score, self.bc, self.wc,
            119 - self.ep if self.ep and not nullmove else 0,
            119 - self.kp if self.kp and not nullmove else 0,
        )

    def move(self, move):
        i, j, prom = move
        p, q = self.board[i], self.board[j]
        put = lambda board, i, p: board[:i] + p + board[i + 1 :]
        # Copy variables and reset ep and kp
        board = self.board
        wc, bc, ep, kp = self.wc, self.bc, 0, 0
        score = self.score + self.value(move)
        # Actual move
        board = put(board, j, board[i])
        board = put(board, i, ".")
        # Castling rights, we move the rook or capture the opponent's
        if i == A1: wc = (False, wc[1])
        if i == H1: wc = (wc[0], False)
        if j == A8: bc = (bc[0], False)
        if j == H8: bc = (False, bc[1])
        # Castling
        if p == "K":
            wc = (False, False)
            if abs(j - i) == 2:
                kp = (i + j) // 2
                board = put(board, A1 if j < i else H1, ".")
                board = put(board, kp, "R")
        # Pawn promotion, double move and en passant capture
        if p == "P":
            if A8 <= j <= H8:
                board = put(board, j, prom)
            if j - i == 2 * N:
                ep = i + N
            if j == self.ep:
                board = put(board, j + S, ".")
        # We rotate the returned position, so it's ready for the next player
        return Position(board, score, wc, bc, ep, kp).rotate()

    def value(self, move):
        i, j, prom = move
        p, q = self.board[i], self.board[j]
        # Actual move
        score = pst[p][j] - pst[p][i]
        # Capture
        if q.islower():
            score += pst[q.upper()][119 - j]
        # Castling check detection
        if abs(j - self.kp) < 2:
            score += pst["K"][119 - j]
        # Castling
        if p == "K" and abs(i - j) == 2:
```

```
            score += pst["R"][(i + j) // 2]
            score -= pst["R"][A1 if j < i else H1]
        # Special pawn stuff
        if p == "P":
            if A8 <= j <= H8:
                score += pst[prom][j] - pst["P"][j]
            if j == self.ep:
                score += pst["P"][119 - (j + S)]
        return score


###############################################################################
# Search logic
###############################################################################

# lower <= s(pos) <= upper
Entry = namedtuple("Entry", "lower upper")


class Searcher:
    def __init__(self):
        self.tp_score = {}
        self.tp_move = {}
        self.history = set()
        self.nodes = 0

    def bound(self, pos, gamma, depth, can_null=True):
        """ Let s* be the "true" score of the sub-tree we are searching.
            The method returns r, where
            if gamma >  s* then s* <= r < gamma  (A better upper bound)
            if gamma <= s* then gamma <= r <= s* (A better lower bound) """
        self.nodes += 1

        # Depth <= 0 is QSearch. Here any position is searched as deeply as is needed for
        # calmness, and from this point on there is no difference in behaviour depending o
        # depth, so so there is no reason to keep different depths in the transposition ta
        depth = max(depth, 0)

        # Sunfish is a king-capture engine, so we should always check if we
        # still have a king. Notice since this is the only termination check,
        # the remaining code has to be comfortable with being mated, stalemated
        # or able to capture the opponent king.
        if pos.score <= -MATE_LOWER:
            return -MATE_UPPER

        # Look in the table if we have already searched this position before.
        # We also need to be sure, that the stored search was over the same
        # nodes as the current search.
        entry = self.tp_score.get((pos, depth, can_null), Entry(-MATE_UPPER, MATE_UPPER))
        if entry.lower >= gamma: return entry.lower
        if entry.upper < gamma: return entry.upper

        # Let's not repeat positions. We don't chat
        # - at the root (can_null=False) since it is in history, but not a draw.
        # - at depth=0, since it would be expensive and break "futility pruning".
        if can_null and depth > 0 and pos in self.history:
            return 0

        # Generator of moves to search in order.
        # This allows us to define the moves, but only calculate them if needed.
        def moves():
            # First try not moving at all. We only do this if there is at least one major
            # piece left on the board, since otherwise zugzwangs are too dangerous.
            # FIXME: We also can't null move if we can capture the opponent king.
```

```
            # Since if we do, we won't spot illegal moves that could lead to stalemate.
            # For now we just solve this by not using null-move in very unbalanced positio
            # TODO: We could actually use null-move in QS as well. Not sure it would be ve
            # But still.... We just have to move stand-pat to be before null-move.
            #if depth > 2 and can_null and any(c in pos.board for c in "RBNQ"):
            #if depth > 2 and can_null and any(c in pos.board for c in "RBNQ") and abs(pos
            if depth > 2 and can_null and abs(pos.score) < 500:
                yield None, -self.bound(pos.rotate(nullmove=True), 1 - gamma, depth - 3)

            # For QSearch we have a different kind of null-move, namely we can just stop
            # and not capture anything else.
            if depth == 0:
                yield None, pos.score

            # Look for the strongest ove from last time, the hash-move.
            killer = self.tp_move.get(pos)

            # If there isn't one, try to find one with a more shallow search.
            # This is known as Internal Iterative Deepening (IID). We set
            # can_null=True, since we want to make sure we actually find a move.
            if not killer and depth > 2:
                self.bound(pos, gamma, depth - 3, can_null=False)
                killer = self.tp_move.get(pos)

            # If depth == 0 we only try moves with high intrinsic score (captures and
            # promotions). Otherwise we do all moves. This is called quiescent search.
            val_lower = QS - depth * QS_A

            # Only play the move if it would be included at the current val-limit,
            # since otherwise we'd get search instability.
            # We will search it again in the main loop below, but the tp will fix
            # things for us.
            if killer and pos.value(killer) >= val_lower:
                yield killer, -self.bound(pos.move(killer), 1 - gamma, depth - 1)

            # Then all the other moves
            for val, move in sorted(((pos.value(m), m) for m in pos.gen_moves()), reverse=
                # Quiescent search
                if val < val_lower:
                    break

                # If the new score is less than gamma, the opponent will for sure just
                # stand pat, since ""pos.score + val < gamma === -(pos.score + val) >= 1-g
                # This is known as futility pruning.
                if depth <= 1 and pos.score + val < gamma:
                    # Need special case for MATE, since it would normally be caught
                    # before standing pat.
                    yield move, pos.score + val if val < MATE_LOWER else MATE_UPPER
                    # We can also break, since we have ordered the moves by value,
                    # so it can't get any better than this.
                    break

                yield move, -self.bound(pos.move(move), 1 - gamma, depth - 1)

        # Run through the moves, shortcutting when possible
        best = -MATE_UPPER
        for move, score in moves():
            best = max(best, score)
            if best >= gamma:
                # Save the move for pv construction and killer heuristic
                if move is not None:
                    self.tp_move[pos] = move
                break

```

```
385         # Stalemate checking is a bit tricky: Say we failed low, because
386         # we can't (legally) move and so the (real) score is -infty.
387         # At the next depth we are allowed to just return r, -infty <= r < gamma,
388         # which is normally fine.
389         # However, what if gamma = -10 and we don't have any legal moves?
390         # Then the score is actaully a draw and we should fail high!
391         # Thus, if best < gamma and best < 0 we need to double check what we are doing.
392
393         # We will fix this problem another way: We add the requirement to bound, that
394         # it always returns MATE_UPPER if the king is capturable. Even if another move
395         # was also sufficient to go above gamma. If we see this value we know we are eithe
396         # mate, or stalemate. It then suffices to check whether we're in check.
397
398         # Note that at low depths, this may not actually be true, since maybe we just prun
399         # all the legal moves. So sunfish may report "mate", but then after more search
400         # realize it's not a mate after all. That's fair.
401
402         # This is too expensive to test at depth == 0
403         if depth > 2 and best == -MATE_UPPER:
404             flipped = pos.rotate(nullmove=True)
405             # Hopefully this is already in the TT because of null-move
406             in_check = self.bound(flipped, MATE_UPPER, 0) == MATE_UPPER
407             best = -MATE_LOWER if in_check else 0
408
409         # Table part 2
410         if best >= gamma:
411             self.tp_score[pos, depth, can_null] = Entry(best, entry.upper)
412         if best < gamma:
413             self.tp_score[pos, depth, can_null] = Entry(entry.lower, best)
414
415         return best
416
417     def search(self, history):
418         """Iterative deepening MTD-bi search"""
419         self.nodes = 0
420         self.history = set(history)
421         self.tp_score.clear()
422
423         gamma = 0
424         # In finished games, we could potentially go far enough to cause a recursion
425         # limit exception. Hence we bound the ply. We also can't start at 0, since
426         # that's quiscent search, and we don't always play legal moves there.
427         for depth in range(1, 1000):
428             # The inner loop is a binary search on the score of the position.
429             # Inv: lower <= score <= upper
430             # 'while lower != upper' would work, but it's too much effort to spend
431             # on what's probably not going to change the move played.
432             lower, upper = -MATE_LOWER, MATE_LOWER
433             while lower < upper - EVAL_ROUGHNESS:
434                 score = self.bound(history[-1], gamma, depth, can_null=False)
435                 if score >= gamma:
436                     lower = score
437                 if score < gamma:
438                     upper = score
439                 yield depth, gamma, score, self.tp_move.get(history[-1])
440                 gamma = (lower + upper + 1) // 2
441
442 # ====================== end of embedded Sunfish engine ======================
443
444 # @manual
445 # @description
446 # PY-16 CHESS (Couch Co-op Edition)
447 # A complete 16-bit chess game for py-16.
448 # Play locally against a friend (with two gamepads)
```

```
# or challenge the built-in chess AI.
# Includes a chess clock, save system, and customizable colors!
#
# @controls
# D-Pad / Arrows    : Move cursor
# Z / A-Button      : Select piece / Execute move
# Mouse / Left click: Alternative controls (Point & Click)
# S / A (Keyboard)  : Save / Load game
# Space / Select    : Back to menu / Cancel game
#
# @credits
# Code, Graphics & UI : py-16 Gamedev
# AI-Engine           : Sunfish (Thomas Ahle)
# @end

# --- GLOBAL STATE VARIABLES ---
board = []
turn = 'w'
selected_sq = None
valid_moves = []
game_status = 'play' # 'play', 'menu', 'settings', 'ai_thinking', 'promote', 'checkmate',
winner = None
vs_ai = False        # Are we playing against the Sunfish AI?

# Variables for advanced chess rules
king_moved = {'w': False, 'b': False}
rook_moved = {'w': [False, False], 'b': [False, False]}
en_passant_target = None
promotion_state = None
was_capture_saved = False

# Undo: stack of full state snapshots, one per played half-move
undo_stack = []
MAX_UNDO = 40               # cap memory; plenty for casual play
pending_move = None         # (sr,sc,r,c) awaiting confirmation when confirm_moves is on

# 50-move rule, threefold repetition and draw bookkeeping
halfmove_clock = 0          # half-moves since last pawn move or capture
position_history = []       # FEN-like keys, for threefold repetition
draw_reason = ""            # text shown when a game ends in a draw

# UI & Settings Variables
show_msg = ""
msg_timer = 0
sound_on = True
difficulty = 1       # 0: Easy, 1: Medium, 2: Hard
confirm_moves = False  # if True, a selected move must be confirmed before it plays

# Time Control Variables
time_options = [0, 60, 300, 600, 900, 1200, 1800, 3600]
time_option_strs = ["NONE", "1:00", "5:00", "10:00", "15:00", "20:00", "30:00", "60:00"]
time_idx = 0
frames_white = 0
frames_black = 0

# Background Variables
bg_options = [13, 1, 2, 3, 5, 12, 0, 4, 8, 11, 14, 15]
bg_names = ["INDIGO", "DARK BLUE", "PURPLE", "GREEN", "GRAY", "BLUE", "BLACK", "BROWN", "R
bg_idx = 0

# Controller Cursor Variables
cursor_r = 4
cursor_c = 4
promote_cursor = 0
```

```
menu_cursor = 0
settings_cursor = 0
demo_timer = 40 # Timer for the background AI in the menu

# Layout Constants
BOARD_X = 64
BOARD_Y = 48
SQ_SIZE = 16

# --- COUCH CO-OP HELPER ---
def get_active_controller():
    """Input source to query for the side whose turn it is.

    Returns 0 ('any source' = keyboard + Player-1 gamepad) for White and for
    pass-and-play, exactly like the menu does, so the keyboard always works
    whether or not a gamepad is plugged in. Only a real second controller in a
    local match gets its own slot 2.
    """
    # Local 2-player with a real second controller: Black is Player 2.
    if turn == 'b' and not vs_ai and py16.player_connected(2):
        return 2
    # Everyone else (White, pass-and-play Black, vs-AI): any source.
    return 0


def get_active_player_label():
    """Player number (1 or 2) to show in the on-screen status line."""
    return 2 if get_active_controller() == 2 else 1

# --- SOUND HELPER ---
def play_sound(*args, **kwargs):
    """Plays a tone, but only if sound is activated."""
    if sound_on:
        py16.tone(*args, **kwargs)

# --- SUNFISH AI: EVALUATION TABLES (PST) ---
SUNFISH_PST = {
    1: [  0,  0,  0,  0,  0,  0,  0,  0,
         78, 83, 86, 73,102, 82, 85, 90,
          7, 29, 21, 44, 40, 31, 44,  7,
        -17, 16, -2, 15, 14,  0, 15,-13,
        -26,  3, 10,  9,  6,  1,  0,-23,
        -22,  9,  5,-11,-10, -2,  3,-19,
        -31,  8, -7,-37,-36,-14,  3,-31,
          0,  0,  0,  0,  0,  0,  0,  0],
    2: [-66,-53,-75,-75,-10,-55,-58,-70,
         -3, -6,100,-36,  4, 62, -4,-14,
         10, 67,  1, 74, 73, 27, 62, -2,
         24, 24, 45, 37, 33, 41, 25, 17,
         -1,  5, 31, 21, 22, 35,  2,  0,
        -18, 10, 13, 22, 18, 15, 11,-14,
        -23,-15,  2,  0,  2,  0,-23,-20,
        -74,-23,-26,-24,-19,-35,-22,-69],
    3: [-59,-78,-82,-76,-23,-107,-37,-50,
        -11, 20, 35,-42,-39, 31,  2,-22,
         -9, 39,-32, 41, 52,-10, 28,-14,
         25, 17, 20, 34, 26, 25, 15, 10,
         13, 10, 17, 23, 17, 16,  0,  7,
         14, 25, 24, 15,  8, 25, 20, 15,
         19, 20, 11,  6,  7,  6, 20, 16,
         -7,  2,-15,-12,-14,-15,-10,-10],
    4: [ 35, 29, 33,  4, 37, 33, 56, 50,
         55, 29, 56, 67, 55, 62, 34, 60,
         19, 35, 28, 33, 45, 27, 25, 15,
```

```
          0,  5, 16, 13, 18, -4, -9, -6,
        -28,-35,-16,-21,-13,-29,-46,-30,
        -42,-28,-42,-25,-25,-35,-26,-46,
        -53,-38,-31,-26,-29,-43,-44,-53,
        -30,-24,-18,  5, -2,-18,-31,-32],
    5: [  6,  1, -8,-104, 69, 24, 88, 26,
         14, 32, 60,-10, 20, 76, 57, 24,
         -2, 43, 32, 60, 72, 63, 43,  2,
          1,-16, 22, 17, 25, 20,-13, -6,
        -14,-15, -2, -5, -1,-10,-20,-22,
        -30, -6,-13,-11,-16,-11,-16,-27,
        -36,-18,  0,-19,-15,-15,-21,-38,
        -39,-30,-31,-13,-31,-36,-34,-42],
    6: [  4, 54, 47,-99,-99, 60, 83,-62,
        -32, 10, 55, 56, 56, 55, 10,  3,
        -62, 12,-57, 44,-67, 28, 37,-31,
        -55, 50, 11, -4,-19, 13,  0,-49,
        -55,-43,-52,-28,-51,-47, -8,-50,
        -47,-42,-43,-79,-64,-32,-29,-32,
         -4,  3,-14,-50,-57,-18, 13,  4,
         17, 30, -3,-14,  6, -1, 40, 18]
}
PIECE_VALUES = {1: 100, 2: 280, 3: 320, 4: 479, 5: 929, 6: 60000}

# --- PIXEL ART DEFINITIONS (16x16) ---
PIXEL_ART = {
    1: [ "                ", "                ", "      ####      ", "     #....#     ", "
    2: [ "                ", "                ", "      ####      ", "     #....##    ", "
    3: [ "                ", "                ", "       ##       ", "      #..#      ", "
    4: [ "                ", "                ", "    #.#..#.#    ", "    #......#    ", "
    5: [ "                ", "                ", "    #...#...#   ", "    #...#...#   ", "
    6: [ "                ", "       ##       ", "      ####      ", "       ##       ", "
}

def generate_sprites():
    """Draws the pixel art dynamically into the py-16 spritesheet."""
    py16.palt(14, True) # 14 (Pink) becomes the transparent mask
    py16.palt(0, False) # Black remains explicitly visible

    for ptype, art in PIXEL_ART.items():
        base_x = (ptype - 1) * 16
        for row_idx, row_str in enumerate(art):
            for col_idx, char in enumerate(row_str):
                cx = base_x + col_idx
                cy_w = row_idx          # Rows 0-15 for White
                cy_b = 16 + row_idx     # Rows 16-31 for Black

                c_w, c_b = 14, 14
                if char == '#':
                    c_w, c_b = 0, 0
                elif char == '.':
                    c_w, c_b = 7, 5

                py16.sset(cx, cy_w, c_w)
                py16.sset(cx, cy_b, c_b)

def reset_board(vs_sunfish=False):
    """Resets the board for a new game."""
    global board, turn, selected_sq, valid_moves, winner, vs_ai
    global king_moved, rook_moved, en_passant_target, promotion_state, was_capture_saved
    global cursor_r, cursor_c, promote_cursor, show_msg, msg_timer
    global frames_white, frames_black
    global halfmove_clock, position_history, draw_reason, undo_stack

```

```
    vs_ai = vs_sunfish
    board = [
        [-4, -2, -3, -5, -6, -3, -2, -4],
        [-1, -1, -1, -1, -1, -1, -1, -1],
        [ 0,  0,  0,  0,  0,  0,  0,  0],
        [ 0,  0,  0,  0,  0,  0,  0,  0],
        [ 0,  0,  0,  0,  0,  0,  0,  0],
        [ 0,  0,  0,  0,  0,  0,  0,  0],
        [ 1,  1,  1,  1,  1,  1,  1,  1],
        [ 4,  2,  3,  5,  6,  3,  2,  4]
    ]
    turn = 'w'
    selected_sq = None
    valid_moves = []
    winner = None

    king_moved = {'w': False, 'b': False}
    rook_moved = {'w': [False, False], 'b': [False, False]}
    en_passant_target = None
    promotion_state = None
    was_capture_saved = False
    cursor_r = 4
    cursor_c = 4
    promote_cursor = 0
    show_msg = ""
    msg_timer = 0
    frames_white = 0
    frames_black = 0
    halfmove_clock = 0
    position_history = []
    draw_reason = ""
    undo_stack = []
    _ai_reset_search()      # drop any in-progress AI search from a previous game
    record_position()

def init():
    """Starting point of the cartridge."""
    global game_status
    reset_board()
    game_status = 'menu'
    generate_sprites()

# --- SAVE / LOAD ---
# One single list of persisted fields, so save() and load() can never drift
# out of sync (the old code maintained two hand-written copies).
SAVE_REQUIRED = ["board", "turn", "king_moved", "rook_moved",
                 "en_passant_target", "game_status", "winner"]
SAVE_OPTIONAL = {
    "vs_ai": False, "sound_on": True, "difficulty": 1, "time_idx": 0,
    "frames_white": 0, "frames_black": 0, "bg_idx": 0,
    "halfmove_clock": 0, "position_history": [], "draw_reason": "",
    "confirm_moves": False,
}
SAVE_FIELDS = SAVE_REQUIRED + list(SAVE_OPTIONAL)


def save_dir():
    """Savegame directory. Lives under ~/.py16/ like the rest of py-16,
    and can be overridden with the PY16_CHESS_SAVE_DIR environment variable."""
    env = os.environ.get("PY16_CHESS_SAVE_DIR")
    if env:
        return os.path.expanduser(env)
    return os.path.expanduser(os.path.join("~", ".py16", "chess_saves"))

```

```
705
706 def _load_candidates():
707     """New location first, then the legacy ./chess_games/ folder for migration."""
708     return [
709         os.path.join(save_dir(), "savegame.json"),
710         os.path.join("chess_games", "savegame.json"),
711     ]
712
713
714 def _validate_state(s):
715     """Raises ValueError if the parsed save is not a usable game state."""
716     for k in SAVE_REQUIRED:
717         if k not in s:
718             raise ValueError("missing field: " + k)
719     b = s["board"]
720     if not isinstance(b, list) or len(b) != 8:
721         raise ValueError("board must have 8 rows")
722     for row in b:
723         if not isinstance(row, list) or len(row) != 8:
724             raise ValueError("board rows must have 8 columns")
725         for v in row:
726             if not isinstance(v, int) or not (-6 <= v <= 6):
727                 raise ValueError("illegal board value")
728     if s["turn"] not in ("w", "b"):
729         raise ValueError("turn must be 'w' or 'b'")
730
731
732 def _flash(msg, freq, wave):
733     global show_msg, msg_timer
734     show_msg = msg
735     msg_timer = 90
736     play_sound(freq, 100, wave)
737
738
739 def save_game():
740     try:
741         d = save_dir()
742         os.makedirs(d, exist_ok=True)
743         state = {k: globals()[k] for k in SAVE_FIELDS}
744         path = os.path.join(d, "savegame.json")
745         tmp = path + ".tmp"
746         with open(tmp, "w") as f:
747             json.dump(state, f)
748         os.replace(tmp, path)          # atomic: never leaves a half-written file
749         _flash("SAVED!", 600, py16.WAVE_SQUARE)
750     except Exception:
751         _flash("SAVE ERROR!", 200, py16.WAVE_SAW)
752
753
754 def load_game():
755     global selected_sq, valid_moves, promotion_state, cursor_r, cursor_c
756
757     state = None
758     for path in _load_candidates():
759         if os.path.exists(path):
760             try:
761                 with open(path, "r") as f:
762                     state = json.load(f)
763                 _validate_state(state)
764                 break
765             except Exception:
766                 _flash("SAVE CORRUPT!", 200, py16.WAVE_SAW)
767                 return
768     if state is None:
```

```
        _flash("NO SAVEGAME!", 200, py16.WAVE_SAW)
        return

    # Commit only after validation succeeded, so a corrupt file can never leave
    # the game in a half-loaded state.
    for k in SAVE_REQUIRED:
        globals()[k] = state[k]
    for k, default in SAVE_OPTIONAL.items():
        globals()[k] = state.get(k, default)

    ep = globals()["en_passant_target"]
    globals()["en_passant_target"] = tuple(ep) if ep else None
    selected_sq = None
    valid_moves = []
    promotion_state = None
    cursor_r, cursor_c = 4, 4

    _flash("GAME LOADED!", 800, py16.WAVE_SAW)


# --- AI SYSTEM (SUNFISH EVALUATION) ---
def evaluate_board(bld):
    """Evaluates the board based on Sunfish material and position tables."""
    score = 0
    for r in range(8):
        for c in range(8):
            p = bld[r][c]
            if p == 0: continue
            ptype = abs(p)
            is_white = (p > 0)

            val = PIECE_VALUES[ptype]

            # Calculate Sunfish PST index
            if is_white:
                idx = r * 8 + c
            else:
                idx = (7 - r) * 8 + c # For Black the evaluation is mirrored

            pst_val = SUNFISH_PST[ptype][idx]

            if is_white:
                score += val + pst_val
            else:
                score -= (val + pst_val)
    return score

def evaluate_move(sr, sc, tr, tc, active_color):
    """Simulates a move on a local board copy and returns the opponent's best
    immediate capture reply (1-Ply + capture-only quiescence). The real game
    board is never touched, which removes a whole class of make/unmake bugs."""
    global en_passant_target
    bld = [row[:] for row in board]
    p = bld[sr][sc]
    bld[tr][tc] = p
    bld[sr][sc] = 0

    # get_pseudo_moves() reads the en-passant target from the module global,
    # so align it with the simulated position for the duration of this call.
    saved_ep = en_passant_target
    if abs(p) == 1 and abs(sr - tr) == 2:
        en_passant_target = ((sr + tr) // 2, sc)
    else:
        en_passant_target = None
```

```
833
834     worst_score = float('inf') if active_color == 'w' else -float('inf')
835     has_captures = False
836     enemy_sign = -1 if active_color == 'w' else 1
837
838     # Strongest opponent capture in the resulting position.
839     for wr in range(8):
840         for wc in range(8):
841             if (bld[wr][wc] * enemy_sign) > 0:
842                 for wtr, wtc in get_pseudo_moves(wr, wc, bld):
843                     target = bld[wtr][wtc]
844                     if (target * enemy_sign) < 0 and abs(target) != 6:
845                         has_captures = True
846                         bld[wtr][wtc] = bld[wr][wc]
847                         bld[wr][wc] = 0
848                         score = evaluate_board(bld)
849                         if active_color == 'w':
850                             worst_score = min(worst_score, score)
851                         else:
852                             worst_score = max(worst_score, score)
853                         bld[wr][wc] = bld[wtr][wtc]
854                         bld[wtr][wtc] = target
855
856     if not has_captures:
857         worst_score = evaluate_board(bld)
858
859     en_passant_target = saved_ep
860     return worst_score
861
862
863 # --- SUNFISH BRIDGE -----------------------------------------------------
864 # Difficulty -> (total time budget in seconds, max search depth).
865 # The search is frame-stepped (see step_sunfish_search), but a single
866 # iterative-deepening step is atomic - Sunfish's generator yields only at the
867 # MTD-bi steps within a depth, so a deep step can briefly stall a frame. That
868 # occasional hitch is accepted in exchange for genuinely strong play.
869 SUNFISH_LEVELS = {0: (0.05, 2), 1: (0.50, 6), 2: (2.50, 24)}
870 # Max seconds of engine work per frame, so the search can be spread across
871 # frames instead of freezing the 60 FPS loop in one big block.
872 AI_FRAME_SLICE = 0.045
873 # Sunfish promotion letter -> cart piece type.
874 _PROM_TO_CART = {'N': 2, 'B': 3, 'R': 4, 'Q': 5, '': 5}
875
876 # In-progress (frame-stepped) search state.
877 _ai_search_gen = None
878 _ai_best_move = None
879 _ai_start_time = 0.0
880 _ai_pos_turn = None
881 _ai_reached_depth = 0
882
883
884 def _cart_to_sunfish_board(cart_board):
885     """Builds Sunfish's 120-char padded board string (White = uppercase)."""
886     rows = ["         \n", "         \n"]
887     for r in range(8):
888         line = " "
889         for c in range(8):
890             v = cart_board[r][c]
891             if v == 0:
892                 line += "."
893             else:
894                 ch = "PNBRQK"[abs(v) - 1]
895                 line += ch if v > 0 else ch.lower()
896         rows.append(line + "\n")
```

```
    rows.append("         \n")
    rows.append("         \n")
    return "".join(rows)


def _sunfish_static_score(board_str):
    """White-perspective sum of piece-square values (matches Sunfish)."""
    s = 0
    for i, p in enumerate(board_str):
        if p.isupper():
            s += pst[p][i]
        elif p.islower():
            s -= pst[p.upper()][119 - i]
    return s


def _cart_to_sunfish_position():
    """Translates the live cart state into a Sunfish Position ready to search."""
    bstr = _cart_to_sunfish_board(board)
    score = _sunfish_static_score(bstr)
    wq = (not king_moved['w']) and (not rook_moved['w'][0]) and board[7][0] == 4
    wk = (not king_moved['w']) and (not rook_moved['w'][1]) and board[7][7] == 4
    bq = (not king_moved['b']) and (not rook_moved['b'][0]) and board[0][0] == -4
    bk = (not king_moved['b']) and (not rook_moved['b'][1]) and board[0][7] == -4
    wc = (bool(wq), bool(wk))      # (queenside/A1, kingside/H1)
    bc = (bool(bk), bool(bq))      # (kingside/H8, queenside/A8) per Position.move
    ep = 0
    if en_passant_target:
        er, ec = en_passant_target
        ep = 21 + er * 10 + ec
    pos = Position(bstr, score, wc, bc, ep, 0)
    if turn == 'b':
        pos = pos.rotate()         # make the side to move 'White' internally
    return pos


def _sunfish_move_to_cart(mv, color):
    """Sunfish Move -> (sr, sc, tr, tc, prom_char), or None if off the board."""
    i, j = mv.i, mv.j
    if color == 'b':               # undo the rotation applied for black
        i, j = 119 - i, 119 - j
    sr, sc = divmod(i - 21, 10)
    tr, tc = divmod(j - 21, 10)
    if not (0 <= sr < 8 and 0 <= sc < 8 and 0 <= tr < 8 and 0 <= tc < 8):
        return None
    return (sr, sc, tr, tc, mv.prom)


def _sunfish_best_move():
    """Synchronous full search (used by tests). Returns (sr, sc, tr, tc) or None."""
    pos = _cart_to_sunfish_position()
    budget, max_depth = SUNFISH_LEVELS.get(difficulty, (0.40, 5))
    searcher = Searcher()
    start = time.time()
    best = None
    for depth, gamma, score, mv in searcher.search([pos]):
        if score >= gamma and mv is not None:
            best = mv
        if best is not None and (depth >= max_depth or time.time() - start > budget):
            break
        if depth >= 40:
            break
    if best is None:
        return None
```

```
    conv = _sunfish_move_to_cart(best, turn)
    if conv is None:
        return None
    sr, sc, tr, tc, _ = conv
    return (sr, sc, tr, tc)


def _play_random_move(color):
    """Plays a random legal move (used to make 'Easy' blunder). True if it moved."""
    import random
    sign = 1 if color == 'w' else -1
    moves = []
    for r in range(8):
        for c in range(8):
            if board[r][c] * sign > 0:
                for tr, tc in get_legal_moves(r, c):
                    moves.append((r, c, tr, tc))
    if not moves:
        return False
    sr, sc, tr, tc = random.choice(moves)
    execute_move(sr, sc, tr, tc)
    return True


def _ai_reset_search():
    global _ai_search_gen, _ai_best_move, _ai_start_time, _ai_pos_turn, _ai_reached_depth
    _ai_search_gen = None
    _ai_best_move = None
    _ai_start_time = 0.0
    _ai_pos_turn = None
    _ai_reached_depth = 0


def _finish_ai_with_fallback():
    """If Sunfish errors out, fall back to the simple built-in evaluator."""
    _ai_reset_search()
    do_ai_turn('b')               # also resets 'ai_thinking' -> 'play'


def _apply_ai_best_move():
    """Convert the chosen Sunfish move, validate it, and play it on the cart."""
    global game_status
    mv = _ai_best_move
    color = _ai_pos_turn
    _ai_reset_search()
    if mv is not None:
        conv = _sunfish_move_to_cart(mv, color)
        if conv is not None:
            sr, sc, tr, tc, prom = conv
            if get_piece_color(board[sr][sc]) == 'b' and (tr, tc) in get_legal_moves(sr, s
                execute_move(sr, sc, tr, tc, ai_promo=_PROM_TO_CART.get(prom, 5))
                if game_status == 'ai_thinking':
                    game_status = 'play'
                return
    do_ai_turn('b')               # fallback


def step_sunfish_search():
    """Advance the AI search by ~one frame of work; play the move once the
    time budget or depth target is reached. Keeps the UI responsive."""
    global _ai_search_gen, _ai_best_move, _ai_start_time, _ai_pos_turn, game_status, _ai_r

    # First frame of a new search: set it up.
    if _ai_search_gen is None:
```

```
        # Easy occasionally blunders like a beginner.
        if difficulty == 0 and py16.rnd(100) < 35:
            if _play_random_move('b'):
                if game_status == 'ai_thinking':
                    game_status = 'play'
                _ai_reset_search()
                return
        try:
            pos = _cart_to_sunfish_position()
            _ai_pos_turn = turn
            _ai_search_gen = Searcher().search([pos])
            _ai_start_time = time.time()
            _ai_best_move = None
        except Exception:
            _finish_ai_with_fallback()
            return

    budget, max_depth = SUNFISH_LEVELS.get(difficulty, (0.30, 4))
    frame_start = time.time()
    done = False
    try:
        while time.time() - frame_start < AI_FRAME_SLICE:
            # Stop before launching another (deeper, more expensive) step once
            # we already have a move and have hit the depth target or time budget.
            if _ai_best_move is not None and (
                    _ai_reached_depth >= max_depth
                    or time.time() - _ai_start_time > budget):
                done = True
                break
            depth, gamma, score, mv = next(_ai_search_gen)
            _ai_reached_depth = depth
            if score >= gamma and mv is not None:
                _ai_best_move = mv
            if depth >= 40:
                done = True
                break
    except StopIteration:
        done = True
    except Exception:
        _finish_ai_with_fallback()
        return

    if done:
        _apply_ai_best_move()


def do_sunfish_turn():
    """Synchronous wrapper around the frame-stepped search (used by tests and
    as a blocking fallback): runs the stepper until the move has been made."""
    _ai_reset_search()
    guard = 0
    while game_status == 'ai_thinking' and guard < 1000000:
        step_sunfish_search()
        guard += 1


def do_ai_turn(color='b'):
    """Executes the AI turn based on Sunfish values and difficulty settings."""
    global game_status
    best_score = -float('inf') if color == 'w' else float('inf')
    best_moves = []
    sign = 1 if color == 'w' else -1

    # Calculate noise level based on difficulty
```

```
    noise_level = 1.0
    if difficulty == 0: noise_level = 200.0 # Easy
    elif difficulty == 1: noise_level = 25.0 # Medium

    for r in range(8):
        for c in range(8):
            if (board[r][c] * sign) > 0:
                moves = get_legal_moves(r, c)
                for tr, tc in moves:
                    score = evaluate_move(r, c, tr, tc, color)
                    # Add random values to simulate mistakes based on difficulty
                    score += (py16.rnd(noise_level * 2) - noise_level)

                    if color == 'w':
                        if score > best_score:
                            best_score = score
                            best_moves = [((r, c), (tr, tc))]
                        elif score == best_score:
                            best_moves.append(((r, c), (tr, tc)))
                    else:
                        if score < best_score:
                            best_score = score
                            best_moves = [((r, c), (tr, tc))]
                        elif score == best_score:
                            best_moves.append(((r, c), (tr, tc)))

    if best_moves:
        import random
        move = random.choice(best_moves)
        execute_move(move[0][0], move[0][1], move[1][0], move[1][1])
    elif game_status == 'menu':
        # Fallback in demo mode if no more moves are possible
        reset_board(False)
        game_status = 'menu'
        return

    if game_status == 'ai_thinking':
        game_status = 'play'

# --- CHESS RULE LOGIC ---
def get_piece_color(val):
    if val > 0: return 'w'
    if val < 0: return 'b'
    return None

def get_pseudo_moves(r, c, bld):
    p = bld[r][c]
    if p == 0: return []
    color = 'w' if p > 0 else 'b'
    ptype = abs(p)
    moves = []

    if ptype == 1:
        dr = -1 if color == 'w' else 1
        nr = r + dr
        if 0 <= nr < 8 and bld[nr][c] == 0:
            moves.append((nr, c))
            start_row = 6 if color == 'w' else 1
            if r == start_row and bld[r + 2*dr][c] == 0:
                moves.append((r + 2*dr, c))
        for dc in [-1, 1]:
            nc = c + dc
            nr = r + dr
            if 0 <= nr < 8 and 0 <= nc < 8:
```

```
                target = bld[nr][nc]
                if target != 0 and get_piece_color(target) != color:
                    moves.append((nr, nc))
        if en_passant_target:
            tr, tc = en_passant_target
            if tr == r + dr and abs(tc - c) == 1:
                moves.append((tr, tc))

    elif ptype == 2:
        offsets = [(2,1), (2,-1), (-2,1), (-2,-1), (1,2), (1,-2), (-1,2), (-1,-2)]
        for dr, dc in offsets:
            nr, nc = r + dr, c + dc
            if 0 <= nr < 8 and 0 <= nc < 8:
                if bld[nr][nc] == 0 or get_piece_color(bld[nr][nc]) != color:
                    moves.append((nr, nc))

    elif ptype in [3, 4, 5]:
        dirs = []
        if ptype == 3 or ptype == 5:
            dirs += [(1,1), (1,-1), (-1,1), (-1,-1)]
        if ptype == 4 or ptype == 5:
            dirs += [(1,0), (-1,0), (0,1), (0,-1)]

        for dr, dc in dirs:
            nr, nc = r + dr, c + dc
            while 0 <= nr < 8 and 0 <= nc < 8:
                target = bld[nr][nc]
                if target == 0:
                    moves.append((nr, nc))
                else:
                    if get_piece_color(target) != color:
                        moves.append((nr, nc))
                    break
                nr += dr
                nc += dc

    elif ptype == 6:
        for dr in [-1, 0, 1]:
            for dc in [-1, 0, 1]:
                if dr == 0 and dc == 0: continue
                nr, nc = r + dr, c + dc
                if 0 <= nr < 8 and 0 <= nc < 8:
                    if bld[nr][nc] == 0 or get_piece_color(bld[nr][nc]) != color:
                        moves.append((nr, nc))
    return moves

def is_in_check(color, bld):
    k_pos = None
    target_king = 6 if color == 'w' else -6
    for r in range(8):
        for c in range(8):
            if bld[r][c] == target_king:
                k_pos = (r, c)
                break
        if k_pos: break
    if not k_pos: return False

    enemy_color = 'b' if color == 'w' else 'w'
    for r in range(8):
        for c in range(8):
            piece = bld[r][c]
            if piece != 0 and get_piece_color(piece) == enemy_color:
                if abs(piece) == 1:
                    dr = -1 if enemy_color == 'w' else 1
```

```
                    if r + dr == k_pos[0] and abs(c - k_pos[1]) == 1:
                        return True
                else:
                    if k_pos in get_pseudo_moves(r, c, bld):
                        return True
    return False

def get_legal_moves(r, c):
    p = board[r][c]
    color = get_piece_color(p)
    pseudo = get_pseudo_moves(r, c, board)
    legal = []

    for tr, tc in pseudo:
        saved_target = board[tr][tc]

        is_ep = (abs(p) == 1 and en_passant_target == (tr, tc))
        if is_ep:
            saved_ep_piece = board[r][tc]
            board[r][tc] = 0

        board[tr][tc] = p
        board[r][c] = 0

        if not is_in_check(color, board):
            legal.append((tr, tc))

        board[r][c] = p
        board[tr][tc] = saved_target
        if is_ep:
            board[r][tc] = saved_ep_piece

    if abs(p) == 6 and not king_moved[color] and not is_in_check(color, board):
        if color == 'w' and r == 7 and c == 4:
            if not rook_moved['w'][1] and board[7][7] == 4 and board[7][5] == 0 and board[
                board[7][5] = 6; board[7][4] = 0
                if not is_in_check('w', board): legal.append((7, 6))
                board[7][4] = 6; board[7][5] = 0
            if not rook_moved['w'][0] and board[7][0] == 4 and board[7][1] == 0 and board[
                board[7][3] = 6; board[7][4] = 0
                if not is_in_check('w', board): legal.append((7, 2))
                board[7][4] = 6; board[7][3] = 0
        elif color == 'b' and r == 0 and c == 4:
            if not rook_moved['b'][1] and board[0][7] == -4 and board[0][5] == 0 and board
                board[0][5] = -6; board[0][4] = 0
                if not is_in_check('b', board): legal.append((0, 6))
                board[0][4] = -6; board[0][5] = 0
            if not rook_moved['b'][0] and board[0][0] == -4 and board[0][1] == 0 and board
                board[0][3] = -6; board[0][4] = 0
                if not is_in_check('b', board): legal.append((0, 2))
                board[0][4] = -6; board[0][3] = 0

    return legal

def has_any_legal_moves(color):
    for r in range(8):
        for c in range(8):
            if board[r][c] != 0 and get_piece_color(board[r][c]) == color:
                if len(get_legal_moves(r, c)) > 0:
                    return True
    return False

def position_key():
    """Compact FEN-like key of the current position for the repetition check."""
```

```
    rows = "|".join(",".join(str(v) for v in board[r]) for r in range(8))
    castle = (
        int(king_moved['w']), int(rook_moved['w'][0]), int(rook_moved['w'][1]),
        int(king_moved['b']), int(rook_moved['b'][0]), int(rook_moved['b'][1])
    )
    ep = en_passant_target if en_passant_target else "-"
    return f"{rows};{turn};{castle};{ep}"


def record_position():
    """Stores the current position key (used for threefold repetition)."""
    position_history.append(position_key())


def is_insufficient_material():
    """True for dead-draw material: K-K, K+minor-K, same-coloured bishops only."""
    knights = 0
    bishops = 0
    bishop_sq_colors = set()
    for r in range(8):
        for c in range(8):
            p = board[r][c]
            if p == 0:
                continue
            t = abs(p)
            if t == 6:
                continue
            if t in (1, 4, 5):
                return False              # pawn, rook or queen -> mate possible
            if t == 2:
                knights += 1
            elif t == 3:
                bishops += 1
                bishop_sq_colors.add((r + c) % 2)
    if knights == 0 and bishops == 0:
        return True                       # K vs K
    if knights + bishops == 1:
        return True                       # K+N vs K  or  K+B vs K
    if knights == 0 and len(bishop_sq_colors) == 1:
        return True                       # only bishops, all on one square colour
    return False


def declare_draw(reason):
    global game_status, winner, draw_reason
    game_status = 'draw'
    winner = 'draw'
    draw_reason = reason
    play_sound(300, 400, py16.WAVE_TRIANGLE)


def check_game_over_conditions():
    global game_status, winner
    if not has_any_legal_moves(turn):
        if is_in_check(turn, board):
            game_status = 'checkmate'
            winner = 'b' if turn == 'w' else 'w'
            play_sound(200, 600, py16.WAVE_SAW)
        else:
            declare_draw("STALEMATE")
        return

    # Automatic draws while the game is still in progress
    if is_insufficient_material():
```

```
        declare_draw("DRAW: MATERIAL")
    elif halfmove_clock >= 100:
        declare_draw("DRAW: 50-MOVE RULE")
    elif position_history and position_history.count(position_history[-1]) >= 3:
        declare_draw("DRAW: REPETITION")


def finalize_turn(is_capture):
    global selected_sq, valid_moves, turn
    selected_sq = None
    valid_moves = []

    if is_capture:
        play_sound(180, 100, py16.WAVE_SAW, decay_ms=40)
    else:
        play_sound(440, 50, py16.WAVE_SQUARE, decay_ms=20)

    turn = 'b' if turn == 'w' else 'w'
    record_position()
    check_game_over_conditions()

def _snapshot_state():
    """A deep-ish copy of everything a move can change, for undo."""
    return {
        "board": [row[:] for row in board],
        "turn": turn,
        "king_moved": {"w": king_moved["w"], "b": king_moved["b"]},
        "rook_moved": {"w": rook_moved["w"][:], "b": rook_moved["b"][:]},
        "en_passant_target": en_passant_target,
        "halfmove_clock": halfmove_clock,
        "position_history": position_history[:],
        "game_status": game_status,
        "winner": winner,
        "draw_reason": draw_reason,
        "frames_white": frames_white,
        "frames_black": frames_black,
    }


def push_undo():
    """Records the current state so the move about to be played can be undone."""
    undo_stack.append(_snapshot_state())
    if len(undo_stack) > MAX_UNDO:
        undo_stack.pop(0)


def can_undo():
    return len(undo_stack) > 0


def undo_move():
    """Reverts the last half-move (or, vs the AI, the last full move so the
    human keeps the move). Returns True if anything was undone."""
    global board, turn, king_moved, rook_moved, en_passant_target
    global halfmove_clock, position_history, game_status, winner, draw_reason
    global frames_white, frames_black, selected_sq, valid_moves, promotion_state

    # Versus the AI, one tap should undo both the AI reply and your own move,
    # otherwise it would just be the AI's turn again.
    steps = 1
    if vs_ai and len(undo_stack) >= 2:
        steps = 2

    s = None
```

```
    for _ in range(steps):
        if not undo_stack:
            break
        s = undo_stack.pop()

    if s is None:
        return False

    board = s["board"]
    turn = s["turn"]
    king_moved = s["king_moved"]
    rook_moved = s["rook_moved"]
    en_passant_target = s["en_passant_target"]
    halfmove_clock = s["halfmove_clock"]
    position_history = s["position_history"]
    game_status = s["game_status"]
    winner = s["winner"]
    draw_reason = s["draw_reason"]
    frames_white = s["frames_white"]
    frames_black = s["frames_black"]
    selected_sq = None
    valid_moves = []
    promotion_state = None
    _ai_reset_search()
    return True


# Standard material point values, indexed by abs(piece type).
_MATERIAL_VALUES = {1: 1, 2: 3, 3: 3, 4: 5, 5: 9, 6: 0}
# Full set each side starts with, for deriving what's been captured.
_FULL_SET = {1: 8, 2: 2, 3: 2, 4: 2, 5: 1, 6: 1}


def captured_by_color():
    """Returns (white_captured, black_captured) where each is a dict
    {piece_type: count} of ENEMY pieces that side has captured."""
    on_board_w = {t: 0 for t in _FULL_SET}
    on_board_b = {t: 0 for t in _FULL_SET}
    for r in range(8):
        for c in range(8):
            v = board[r][c]
            if v > 0:
                on_board_w[v] += 1
            elif v < 0:
                on_board_b[-v] += 1
    # White captured = black pieces missing from the board, and vice versa.
    white_captured = {t: _FULL_SET[t] - on_board_b[t] for t in _FULL_SET}
    black_captured = {t: _FULL_SET[t] - on_board_w[t] for t in _FULL_SET}
    return white_captured, black_captured


def material_balance():
    """Positive = White is ahead by this many points, negative = Black ahead."""
    score = 0
    for r in range(8):
        for c in range(8):
            v = board[r][c]
            if v != 0:
                pts = _MATERIAL_VALUES[abs(v)]
                score += pts if v > 0 else -pts
    return score


def find_king(color):
```

```
    """(row, col) of the given side's king, or None."""
    target = 6 if color == 'w' else -6
    for r in range(8):
        for c in range(8):
            if board[r][c] == target:
                return (r, c)
    return None


def execute_move(sr, sc, r, c, ai_promo=None):
    """Central function for executing and documenting a move."""
    global was_capture_saved, en_passant_target, promotion_state, game_status, promote_cur

    # Record undo state for every real game move (not the menu background demo).
    if game_status != 'menu':
        push_undo()

    p = board[sr][sc]
    was_capture_saved = board[r][c] != 0 or (abs(p) == 1 and en_passant_target == (r, c))

    # 50-move rule: reset on captures and pawn moves, otherwise count up
    if abs(p) == 1 or was_capture_saved:
        halfmove_clock = 0
    else:
        halfmove_clock += 1

    if abs(p) == 1 and en_passant_target == (r, c):
        board[sr][c] = 0

    if abs(p) == 6 and abs(sc - c) == 2:
        if c == 6:
            board[r][5] = board[r][7]
            board[r][7] = 0
        elif c == 2:
            board[r][3] = board[r][0]
            board[r][0] = 0

    board[r][c] = board[sr][sc]
    board[sr][sc] = 0

    if abs(p) == 6:
        king_moved[turn] = True
    if abs(p) == 4:
        if sr == 7:
            if sc == 0: rook_moved['w'][0] = True
            if sc == 7: rook_moved['w'][1] = True
        elif sr == 0:
            if sc == 0: rook_moved['b'][0] = True
            if sc == 7: rook_moved['b'][1] = True

    next_ep_target = None
    if abs(p) == 1 and abs(sr - r) == 2:
        next_ep_target = ((sr + r) // 2, sc)
    en_passant_target = next_ep_target

    if abs(p) == 1 and (r == 0 or r == 7):
        promotion_state = (r, c)
        promote_cursor = 0
        if game_status == 'menu' or game_status == 'ai_thinking' or (vs_ai and turn == 'b'
            ptype = ai_promo if ai_promo else 5   # honour AI underpromotion, else queen
            board[r][c] = ptype if p > 0 else -ptype
            promotion_state = None
            finalize_turn(was_capture_saved)
        else:
```

```
            game_status = 'promote'
    else:
        finalize_turn(was_capture_saved)


# --- MAIN LOOP ---
def update():
    global selected_sq, valid_moves, turn, game_status, board, sound_on, difficulty
    global cursor_r, cursor_c, promote_cursor, msg_timer, menu_cursor, settings_cursor, de
    global time_idx, frames_white, frames_black, bg_idx, winner
    global pending_move, confirm_moves

    if msg_timer > 0:
        msg_timer -= 1

    if game_status == 'menu':
        # In the menu we listen for any input (player=0)
        if py16.btnp('up', player=0):
            menu_cursor = max(0, menu_cursor - 1)
            play_sound(440, 30, py16.WAVE_TRIANGLE)
        if py16.btnp('down', player=0):
            menu_cursor = min(4, menu_cursor + 1)
            play_sound(440, 30, py16.WAVE_TRIANGLE)

        # Hybrid mouse support in the menu
        if py16.mouse_btnp(0):
            mx, my = py16.mouse_x(), py16.mouse_y()
            if 40 <= mx <= 216:
                if 90 <= my <= 110:
                    menu_cursor = 0
                    reset_board(True)
                    game_status = 'play'
                    play_sound(600, 50, py16.WAVE_SQUARE)
                    return
                elif 110 <= my <= 130:
                    menu_cursor = 1
                    reset_board(False)
                    game_status = 'play'
                    play_sound(600, 50, py16.WAVE_SQUARE)
                    return
                elif 130 <= my <= 150:
                    menu_cursor = 2
                    load_game()
                    play_sound(600, 50, py16.WAVE_SQUARE)
                    return
                elif 150 <= my <= 170:
                    menu_cursor = 3
                    game_status = 'settings'
                    play_sound(600, 50, py16.WAVE_SQUARE)
                    return
                elif 170 <= my <= 190:
                    menu_cursor = 4
                    game_status = 'license'
                    play_sound(600, 50, py16.WAVE_SQUARE)
                    return

        # Confirm
        if py16.btnp('z', player=0) or py16.btnp('enter', player=0):
            if menu_cursor == 0:
                reset_board(True)
                game_status = 'play'
            elif menu_cursor == 1:
                reset_board(False)
                game_status = 'play'
```

```
            elif menu_cursor == 2:
                load_game()
            elif menu_cursor == 3:
                game_status = 'settings'
            elif menu_cursor == 4:
                game_status = 'license'
            play_sound(600, 50, py16.WAVE_SQUARE)
            return

        # --- DEMO AI ---
        demo_timer -= 1
        if demo_timer <= 0:
            demo_timer = 40 # One move every 40 frames
            do_ai_turn(turn)
            if game_status in ['checkmate', 'stalemate', 'draw']:
                reset_board(False)
                game_status = 'menu'
        return

    if game_status == 'settings':
        if py16.btnp('up', player=0):
            settings_cursor = max(0, settings_cursor - 1)
            play_sound(440, 30, py16.WAVE_TRIANGLE)
        if py16.btnp('down', player=0):
            settings_cursor = min(5, settings_cursor + 1)
            play_sound(440, 30, py16.WAVE_TRIANGLE)

        dir = 0
        if py16.btnp('left', player=0): dir = -1
        elif py16.btnp('right', player=0): dir = 1

        action = py16.btnp('z', player=0) or py16.btnp('enter', player=0)

        if py16.mouse_btnp(0):
            mx, my = py16.mouse_x(), py16.mouse_y()
            if 40 <= mx <= 216:
                if 90 <= my <= 108: settings_cursor = 0; dir = 1
                elif 108 <= my <= 126: settings_cursor = 1; dir = 1
                elif 126 <= my <= 144: settings_cursor = 2; dir = 1
                elif 144 <= my <= 162: settings_cursor = 3; dir = 1
                elif 162 <= my <= 180: settings_cursor = 4; dir = 1
                elif 180 <= my <= 198: settings_cursor = 5; action = True

        # Toggle settings or press back button
        if dir != 0 or action:
            if dir == 0 and action: dir = 1 # Z-Key acts as a right arrow step

            if settings_cursor == 0:
                difficulty = (difficulty + dir) % 3
                play_sound(440, 30, py16.WAVE_TRIANGLE)
            elif settings_cursor == 1:
                time_idx = (time_idx + dir) % 8
                play_sound(440, 30, py16.WAVE_TRIANGLE)
            elif settings_cursor == 2:
                sound_on = not sound_on
                play_sound(440, 30, py16.WAVE_TRIANGLE)
            elif settings_cursor == 3:
                bg_idx = (bg_idx + dir) % len(bg_options)
                play_sound(440, 30, py16.WAVE_TRIANGLE)
            elif settings_cursor == 4:
                confirm_moves = not confirm_moves
                play_sound(440, 30, py16.WAVE_TRIANGLE)
            elif settings_cursor == 5 and action:
                game_status = 'menu'
```

```
                play_sound(600, 50, py16.WAVE_SQUARE)
        return

    if game_status == 'license':
        if py16.btnp('z', player=0) or py16.btnp('space', player=0) or py16.btnp('enter',
            game_status = 'menu'
            play_sound(600, 50, py16.WAVE_SQUARE)
        return

    # --- TIME CONTROL LOGIC ---
    if game_status == 'play' and time_idx > 0:
        if turn == 'w':
            frames_white += 1
            if frames_white >= time_options[time_idx] * 60:
                game_status = 'checkmate'
                winner = 'b'
                play_sound(200, 600, py16.WAVE_SAW)
        elif turn == 'b' and not (vs_ai and game_status == 'ai_thinking'):
            frames_black += 1
            if frames_black >= time_options[time_idx] * 60:
                game_status = 'checkmate'
                winner = 'w'
                play_sound(200, 600, py16.WAVE_SAW)

    # Trigger for AI moves
    if game_status == 'play':
        if vs_ai and turn == 'b':
            game_status = 'ai_thinking'
            _ai_reset_search()      # start a fresh, frame-stepped search
            return # Renders the "AI THINKING..." label in the draw call

    if game_status == 'ai_thinking':
        step_sunfish_search()   # one frame's slice; plays the move when ready
        return

    # Saving and loading can be triggered by anyone (player=0)
    if py16.btnp('s', player=0):
        save_game()
        return
    if py16.btnp('a', player=0):
        load_game()
        return

    # Undo the last move (X). Allowed during play or after a finished game.
    if py16.btnp('x', player=0):
        if game_status in ['play', 'checkmate', 'stalemate', 'draw'] and can_undo():
            if undo_move():
                pending_move = None
                play_sound(500, 60, py16.WAVE_TRIANGLE)
            else:
                play_sound(200, 80, py16.WAVE_SAW)
        else:
            play_sound(200, 80, py16.WAVE_SAW)
        return

    # Back to menu
    if game_status in ['checkmate', 'stalemate', 'draw']:
        if py16.btnp('space', player=0) or py16.btnp('enter', player=0):
            game_status = 'menu'
            return
    else:
        if py16.btnp('space', player=0):
            game_status = 'menu'
            return
```

```
1729
1730      # Menu for pawn promotion (Uses the active controller)
1731      if game_status == 'promote':
1732          cp = get_active_controller()
1733          if py16.btnp('left', player=cp): promote_cursor = max(0, promote_cursor - 1)
1734          if py16.btnp('right', player=cp): promote_cursor = min(3, promote_cursor + 1)
1735
1736          action = False
1737          if py16.btnp('z', player=cp) or py16.btnp('enter', player=cp):
1738              action = True
1739          elif py16.mouse_btnp(0):
1740              mx = py16.mouse_x()
1741              my = py16.mouse_y()
1742              bx = BOARD_X
1743              by = BOARD_Y + 48
1744              if bx <= mx < bx + 128 and by <= my < by + 32:
1745                  promote_cursor = (mx - bx) // 32
1746                  action = True
1747
1748          if action:
1749              pr, pc = promotion_state
1750              p_color = 1 if pr == 0 else -1
1751              types = [5, 4, 3, 2]
1752              board[pr][pc] = types[promote_cursor] * p_color
1753
1754              game_status = 'play'
1755              promotion_state = None
1756              finalize_turn(was_capture_saved)
1757          return
1758
1759      if game_status != 'play':
1760          return
1761
1762      # Controller mapping for the current game
1763      cp = get_active_controller()
1764
1765      if py16.btnp('left', player=cp): cursor_c = max(0, cursor_c - 1); play_sound(800, 10,
1766      if py16.btnp('right', player=cp): cursor_c = min(7, cursor_c + 1); play_sound(800, 10,
1767      if py16.btnp('up', player=cp): cursor_r = max(0, cursor_r - 1); play_sound(800, 10, py
1768      if py16.btnp('down', player=cp): cursor_r = min(7, cursor_r + 1); play_sound(800, 10,
1769
1770      action = False
1771      r, c = cursor_r, cursor_c
1772
1773      # Action button
1774      if py16.btnp('z', player=cp) or py16.btnp('enter', player=cp):
1775          action = True
1776      elif py16.mouse_btnp(0):
1777          mx = py16.mouse_x()
1778          my = py16.mouse_y()
1779          if BOARD_X <= mx < BOARD_X + 128 and BOARD_Y <= my < BOARD_Y + 128:
1780              c = (mx - BOARD_X) // SQ_SIZE
1781              r = (my - BOARD_Y) // SQ_SIZE
1782              cursor_r, cursor_c = r, c
1783              action = True
1784
1785      if action:
1786          if selected_sq and (r, c) in valid_moves:
1787              sr, sc = selected_sq
1788              if confirm_moves and pending_move != (sr, sc, r, c):
1789                  # First press selects the destination; show it and wait for a
1790                  # second press on the same square to confirm.
1791                  pending_move = (sr, sc, r, c)
1792                  play_sound(700, 30, py16.WAVE_TRIANGLE)
```

```
            else:
                pending_move = None
                execute_move(sr, sc, r, c)
        else:
            pending_move = None
            piece = board[r][c]
            if piece != 0 and get_piece_color(piece) == turn:
                selected_sq = (r, c)
                valid_moves = get_legal_moves(r, c)
                play_sound(600, 30, py16.WAVE_TRIANGLE)
            else:
                selected_sq = None
                valid_moves = []

def draw_piece(p, x, y):
    """Draws a chess piece from memory."""
    if p == 0: return
    ptype = abs(p)
    is_black = p < 0
    kx = (ptype - 1) * 2
    ky = 2 if is_black else 0
    spr_id = ky * 32 + kx
    py16.spr(spr_id, x, y, w=2, h=2)

def draw():
    py16.cls(bg_options[bg_idx])

    # --- BACKGROUND & CHESSBOARD (ALWAYS DRAWN) ---
    py16.rect(BOARD_X - 2, BOARD_Y - 2, 132, 132, 0)

    files = "ABCDEFGH"
    for i in range(8):
        py16.text(files[i], BOARD_X + i * 16 + 5, BOARD_Y - 10, 6)
        py16.text(str(8 - i), BOARD_X - 10, BOARD_Y + i * 16 + 5, 6)

    for r in range(8):
        for c in range(8):
            lx = BOARD_X + c * SQ_SIZE
            ly = BOARD_Y + r * SQ_SIZE

            sq_color = 6 if (r + c) % 2 == 0 else 4
            py16.rectfill(lx, ly, SQ_SIZE, SQ_SIZE, sq_color)

            if selected_sq == (r, c) and game_status not in ['menu', 'settings']:
                py16.rect(lx, ly, SQ_SIZE, SQ_SIZE, 10)
                py16.rect(lx + 1, ly + 1, SQ_SIZE - 2, SQ_SIZE - 2, 10)

            if (r, c) in valid_moves and game_status not in ['menu', 'settings']:
                py16.circfill(lx + 8, ly + 8, 2, 11)

            draw_piece(board[r][c], lx, ly)

    # --- CHECK INDICATOR ---
    # Highlight the king of whichever side is currently in check.
    if game_status in ['play', 'promote', 'checkmate']:
        for kcolor in ('w', 'b'):
            if is_in_check(kcolor, board):
                ksq = find_king(kcolor)
                if ksq:
                    kr, kc = ksq
                    klx = BOARD_X + kc * SQ_SIZE
                    kly = BOARD_Y + kr * SQ_SIZE
                    # Pulsing red frame (colour 8) around the checked king.
                    if (py16.t() // 12) % 2 == 0:
```

```
                        py16.rect(klx, kly, SQ_SIZE, SQ_SIZE, 8)
                        py16.rect(klx + 1, kly + 1, SQ_SIZE - 2, SQ_SIZE - 2, 8)

    # --- DRAW CLOCKS ---
    if time_idx > 0:
        # Draw white clock
        rem_w = max(0, time_options[time_idx] - (frames_white // 60))
        min_w, sec_w = rem_w // 60, rem_w % 60
        col_w = 7 if turn == 'w' and game_status in ['play', 'promote'] else 5
        py16.text(f"{min_w}:{sec_w:02d}", BOARD_X + 135, BOARD_Y + 110, col_w)

        # Draw black clock
        rem_b = max(0, time_options[time_idx] - (frames_black // 60))
        min_b, sec_b = rem_b // 60, rem_b % 60
        col_b = 7 if turn == 'b' and game_status in ['play', 'promote', 'ai_thinking'] els
        py16.text(f"{min_b}:{sec_b:02d}", BOARD_X + 135, BOARD_Y + 10, col_b)

    # --- MENU OVERLAYS ---
    if game_status in ['menu', 'settings', 'license']:
        # Slightly darken the screen
        py16.blend_mode("alpha", alpha=160)
        py16.rectfill(0, 0, 256, 224, 0)
        py16.blend_mode("normal")

        if game_status != 'license':
            py16.text("PY-16 CHESS", 90, 64, 7)

        if game_status == 'menu':
            col_ai  = 10 if menu_cursor == 0 else 6
            col_vs  = 10 if menu_cursor == 1 else 6
            col_re  = 10 if menu_cursor == 2 else 6
            col_set = 10 if menu_cursor == 3 else 6
            col_lic = 10 if menu_cursor == 4 else 6

            py16.text("NEW GAME (VS AI)", 58, 100, col_ai)
            py16.text("NEW GAME (LOCAL)", 58, 120, col_vs)
            py16.text("LOAD GAME", 58, 140, col_re)
            py16.text("SETTINGS", 58, 160, col_set)
            py16.text("LICENSE", 58, 180, col_lic)

            cy = 100 + menu_cursor * 20
            py16.text(">", 46, cy, 10)

        elif game_status == 'settings':
            col_dif = 10 if settings_cursor == 0 else 6
            col_tim = 10 if settings_cursor == 1 else 6
            col_snd = 10 if settings_cursor == 2 else 6
            col_bg  = 10 if settings_cursor == 3 else 6
            col_cfm = 10 if settings_cursor == 4 else 6
            col_bck = 10 if settings_cursor == 5 else 6

            diff_strs = ["EASY", "MEDIUM", "HARD"]
            py16.text("AI STRENGTH: " + diff_strs[difficulty], 58, 99, col_dif)

            py16.text("TIME LIMIT: " + time_option_strs[time_idx], 58, 117, col_tim)

            snd_txt = "SOUND: ON" if sound_on else "SOUND: OFF"
            py16.text(snd_txt, 58, 135, col_snd)

            py16.text("BACKGROUND: " + bg_names[bg_idx], 58, 153, col_bg)

            cfm_txt = "CONFIRM MOVES: ON" if confirm_moves else "CONFIRM MOVES: OFF"
            py16.text(cfm_txt, 58, 171, col_cfm)

```

```
            py16.text("BACK", 58, 189, col_bck)

            cy = 99 + settings_cursor * 18
            py16.text(">", 46, cy, 10)

        elif game_status == 'license':
            py16.text("LICENSE INFO", 88, 40, 10)

            py16.text("AI ENGINE:", 40, 70, 6)
            py16.text("SUNFISH (THOMAS AHLE)", 40, 85, 7)

            py16.text("PUBLISHED UNDER:", 40, 115, 6)
            py16.text("GNU GENERAL PUBLIC LICENSE V3", 40, 130, 7)

            py16.text("PRESS ANY KEY TO RETURN", 52, 190, 8)

        if msg_timer > 0 and game_status != 'license':
            col = 8 if (py16.t() // 10) % 2 == 0 else 7
            py16.text(show_msg, 72, 200, col)
        return

    # --- NORMAL GAME UI ---
    py16.text("PY-16 CHESS", 100, 12, 7)

    if msg_timer > 0:
        col = 10 if (py16.t() // 10) % 2 == 0 else 7
        py16.text(show_msg, 92, 36, col)

    if game_status == 'ai_thinking':
        py16.text("AI THINKING...", 84, 26, 8)
    elif game_status == 'play' or game_status == 'promote':
        p_str = f"(P{get_active_player_label()})"
        status_text = f"WHITE TO MOVE {p_str}" if turn == 'w' else f"BLACK TO MOVE {p_str}
        py16.text(status_text, 76, 26, 10 if turn == 'w' else 0)
        if game_status == 'play' and is_in_check(turn, board):
            chk_col = 8 if (py16.t() // 12) % 2 == 0 else 7
            py16.text("CHECK!", 112, 36, chk_col)
    elif game_status == 'checkmate':
        win_text = "MATE! WHITE WINS" if winner == 'w' else "MATE! BLACK WINS"
        py16.text(win_text, 76, 26, 8)
        py16.text("ENTER FOR MENU", 88, 190, 6)
    elif game_status == 'draw':
        py16.text(draw_reason if draw_reason else "DRAW", 72, 26, 14)
        py16.text("ENTER FOR MENU", 88, 190, 6)
    elif game_status == 'stalemate':
        py16.text("STALEMATE! DRAW", 80, 26, 14)
        py16.text("ENTER FOR MENU", 88, 190, 6)

    # --- CAPTURED PIECES & MATERIAL BALANCE ---
    if game_status in ['play', 'promote', 'checkmate', 'stalemate', 'draw']:
        white_capt, black_capt = captured_by_color()
        order = [5, 4, 3, 2, 1]          # Q R B N P
        letters = {1: "P", 2: "N", 3: "B", 4: "R", 5: "Q"}

        def capt_str(capt):
            parts = []
            for t in order:
                if capt[t] > 0:
                    parts.append(letters[t] * capt[t] if capt[t] <= 3 else f"{letters[t]}x
            return "".join(parts)

        # White's captures sit just below the board, black's just above it.
        wy = BOARD_Y + 128 + 2
        by = BOARD_Y - 20
```

```
        py16.text(capt_str(white_capt), BOARD_X, wy, 7)
        py16.text(capt_str(black_capt), BOARD_X, by, 5)

        bal = material_balance()
        if bal != 0:
            who = "W" if bal > 0 else "B"
            py16.text(f"{who}+{abs(bal)}", BOARD_X + 100, BOARD_Y + 128 + 2, 11)

    py16.text("S:SAVE A:LOAD X:UNDO", BOARD_X - 4, BOARD_Y + 138, 6)
    py16.text("SPACE: TO MENU", BOARD_X + 24, BOARD_Y + 148, 6)

    # Controller Cursor
    if game_status == 'play':
        cx = BOARD_X + cursor_c * SQ_SIZE
        cy = BOARD_Y + cursor_r * SQ_SIZE
        blink_col = 8 if (py16.t() // 15) % 2 == 0 else 9
        py16.rect(cx, cy, SQ_SIZE, SQ_SIZE, blink_col)
        py16.rect(cx - 1, cy - 1, SQ_SIZE + 2, SQ_SIZE + 2, blink_col)

        # Pending-move confirmation marker (when CONFIRM MOVES is on).
        if confirm_moves and pending_move is not None:
            _, _, ptr, ptc = pending_move
            px = BOARD_X + ptc * SQ_SIZE
            py = BOARD_Y + ptr * SQ_SIZE
            mark_col = 11 if (py16.t() // 10) % 2 == 0 else 7
            py16.rect(px, py, SQ_SIZE, SQ_SIZE, mark_col)
            py16.rect(px + 1, py + 1, SQ_SIZE - 2, SQ_SIZE - 2, mark_col)
            py16.text("CONFIRM?", 108, 36, mark_col)

    # Promotion Menu
    if game_status == 'promote':
        bx = BOARD_X
        by = BOARD_Y + 48
        py16.rectfill(bx, by, 128, 32, 5)
        py16.rect(bx, by, 128, 32, 7)
        py16.text("CHOOSE PIECE:", bx + 4, by - 10, 7)

        pr, pc = promotion_state
        p_color = 1 if pr == 0 else -1
        types = [5, 4, 3, 2]

        for i in range(4):
            ox = bx + i * 32
            py16.rect(ox, by, 32, 32, 7)
            draw_piece(types[i] * p_color, ox + 8, by + 8)

        px = bx + promote_cursor * 32
        blink_col = 8 if (py16.t() // 15) % 2 == 0 else 9
        py16.rect(px, by, 32, 32, blink_col)
        py16.rect(px - 1, by - 1, 34, 34, blink_col)

if __name__ == "__main__":
    py16.run(update, draw, init)

```